Chapter 1

Introduction to Reinforcement Learning

We're about to make a sharp turn. Up to this point we have written programs that compute things — a temperature trajectory, a polynomial fit, the trajectory of a baseball. In every case we knew, in advance, what the program was supposed to do; the work was figuring out how. In this chapter we begin a different kind of programming: writing a program that *figures out for itself* what to do.

Reinforcement learning (RL) is a computational approach to learning by interaction. An *agent* — the program we are designing — takes actions in an *environment* — the world, real or simulated, that the agent acts on. The environment responds with two things: a new *state* (its updated condition) and a numerical *reward*. The agent's task is to choose actions that maximize the cumulative reward over time. No teacher tells the agent which action is correct; the only feedback is the reward. The agent has to discover, through trial and error, which actions yield the most reward in which states.

The bulk of the material in this chapter is adapted from Sutton and Barto's textbook, *Reinforcement Learning: An Introduction*[1] — the standard reference for the field. We barely scratch the surface; the goal here is to give you enough vocabulary and intuition to implement a small RL system in the next chapter.

1.1 What Reinforcement Learning Is

You may have heard of two other branches of machine learning:

- *Supervised learning* learns from labeled examples. You give the algorithm a set of input-output pairs (e.g., images labeled with the object in the image), and it learns a function that maps inputs to outputs. Curve fitting, as we saw it in the previous chapter, is a kind of supervised learning: given (t, V) pairs, find a function $V(t)$.
- *Unsupervised learning* learns structure from unlabeled data — finding clusters in a

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd edition, MIT Press, 2018. Available online at `http://incompleteideas.net/book/the-book-2nd.html`.

> dataset, or compressing high-dimensional data into a few principal components. There is no explicit "correct answer" for any particular input.

Reinforcement learning is different from both. There are no labeled examples, so it isn't supervised. But there is a clear objective — maximize the cumulative reward — so it is not purely unsupervised either. The defining feature is *interaction*: the agent chooses actions, the environment changes in response, and the agent learns from the consequences.

The classic informal characterization is from Sutton and Barto:

> Reinforcement learning is learning what to do — how to map situations to actions — so as to maximize a numerical reward signal. The learner is not told which actions to take, but instead must discover which actions yield the most reward by trying them. In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situation and, through that, all subsequent rewards.

Two characteristics distinguish RL from other learning paradigms. First, the feedback comes from *trial and error* rather than from labeled examples. The agent has to try an action to learn what reward it produces. Second, the rewards may be *delayed*. An action taken now may not produce a reward until many time steps later, and the agent has to figure out that the early action was responsible.

1.2 Examples

A few examples will help fix ideas.

- A chess player chooses a move. The move is influenced by planning (anticipating the opponent's replies) and by intuition about the desirability of particular positions. The reward, in some sense, comes only at the end of the game — a win or a loss.
- A mobile robot decides whether to enter a new room in search of more trash to collect, or start back to its battery-recharging station. It bases the decision on its current charge and on how easily it has found the recharger in the past.
- A gazelle calf struggles to its feet minutes after birth. Half an hour later it is running at 20 miles per hour.
- An adaptive controller adjusts the operating parameters of a petroleum refinery in real time, optimizing the yield/cost/quality trade-off without sticking strictly to set points originally specified by an engineer.

These examples share a common skeleton. A goal-directed agent interacts with an uncertain environment. The agent's actions can affect future states of the environment, and so the agent has to take long-term consequences into account when deciding what to do now. And in every case, the agent improves with experience.

1.3 The Exploration–Exploitation Trade-off

One distinctive challenge in RL is the trade-off between *exploration* and *exploitation*. The agent has tried a few actions in a few states and built up some experience. In a given state, what should it do next?

It can *exploit*: pick the action that, based on its current experience, looks the best. This is the locally rational choice. But the agent's estimates of how good each action is are based on a small sample, so its current "best" may not actually be optimal.

Or it can *explore*: try an action it hasn't tried much, to gather more information about that action's value. Exploration sometimes finds an even better action than the agent's current best.

Neither strategy alone works well. Pure exploitation never tries new actions, so it gets stuck on whatever first looked promising. Pure exploration never uses what it has learned, so it never collects much reward. An RL agent has to do both.

A common compromise is ε-*greedy* action selection: most of the time (probability $1-\varepsilon$) take the action that looks best, but with small probability ε pick uniformly at random. This guarantees that every action is tried infinitely often in the limit, while spending most of the time on the apparently best action.

The exploration–exploitation dilemma does not arise in supervised or unsupervised learning, where the data are given and fixed. It arises only when the learner's actions affect the data it gets to learn from.

1.4 Elements of a Reinforcement Learning System

Beyond the agent and the environment, an RL system has up to four sub-elements.

Policy. A *policy* is the agent's way of behaving — a mapping from states of the environment to actions. In a simple case, the policy is a lookup table: in state s, take action $\pi(s)$. In more complicated cases, the policy may involve search, planning, or even another learned function. Policies may be *deterministic* (a fixed action per state) or *stochastic* (a probability distribution over actions per state).

Reward signal. On each time step the environment delivers a single number, the *reward*, which the agent is trying to maximize over the long run. The reward signal defines what is *good* in an immediate sense. Designing a reward signal is itself an art: a poorly chosen reward can lead the agent to "game" the system rather than do what you actually wanted.

Value function. A *value function* predicts *long-run* reward. The value of a state is the total reward an agent can expect to accumulate, starting from that state and following its policy thereafter. Whereas the reward says what is good *now*, the value function says what is

good *in the long run*. Most RL algorithms work by estimating value functions and improving them as more experience accumulates.

Model (optional). A *model* of the environment predicts what the environment will do in response to the agent's actions: "if I'm in state s and take action a, the environment transitions to state s' and gives reward r." Models support *planning* — thinking ahead by simulating actions before actually taking them. Methods that use a model are *model-based*; methods that learn purely from experience without a model are *model-free*. Both kinds are useful.

1.5 An Extended Example: Tic-Tac-Toe

To make these ideas concrete, consider the familiar children's game of tic-tac-toe (also called noughts and crosses). Two players take turns placing Xs and Os on a 3-by-3 grid. The first to get three in a row — horizontally, vertically, or diagonally — wins. If the board fills up with neither side winning, the game is a draw.

How might we build a player that learns to play well by experience, without us having to program in the rules of strategy?

The reinforcement-learning approach: associate a number with every reachable board position, representing our current estimate of the probability that we win starting from that position. Call this number the *value* of the state. A position with three Xs in a row has value 1 (we already won); a position with three Os in a row, or a full board with no winner, has value 0 (we cannot win from here). All other positions start with the same initial estimate, say 0.5.

We then play many games against an opponent. On each of our turns, we look at the values of the positions reachable by our possible moves and, most of the time, pick the move that leads to the highest-valued position — a *greedy* move. Occasionally, with small probability, we pick uniformly at random instead — an *exploratory* move.

While we play, we update the value of each state we visit. After each greedy move, we look at the value of the next state and "back up" that value to the state we were in. Specifically, if $V(S_t)$ is our current estimate of the value of state S_t, and S_{t+1} is the state we ended up in after our greedy move, we update

$$V(S_t) \leftarrow V(S_t) + \alpha\big[V(S_{t+1}) - V(S_t)\big],$$

where α is a small positive constant called the *step-size parameter* — it controls how fast we change our estimates. If $V(S_{t+1})$ is larger than $V(S_t)$, then S_t looks better than we thought, and we increase $V(S_t)$. If $V(S_{t+1})$ is smaller, we decrease $V(S_t)$. Over many games, this update rule causes value estimates to converge to good predictions of winning probability.

This procedure — now known as *temporal-difference learning* — has several remarkable properties. It learns just from playing, without any explicit search over future positions. It

does not require a model of the opponent. It improves continually with experience, even against an opponent whose play changes over time (as long as we don't reduce α to zero). And the same idea, with small modifications, scales to much larger problems — including games like chess and Go that have astronomically more states than tic-tac-toe.

We will see in the next chapter that the *Q-learning* algorithm we use for blackjack has exactly this temporal-difference flavor, except that it learns the value of *state-action pairs* rather than just states. This small change makes the method much more useful for problems where we don't have a model of the environment to look ahead with.

1.6 What's Next

The next chapter takes these ideas and turns them into a concrete MATLAB program. We will use the `matlab_gymnasium` package, which provides a simulated blackjack environment, and implement the Q-learning algorithm to train an agent to play. Along the way you will see the `dictionary` data type used to store value estimates indexed by state, and you will get a hands-on feel for how an exploration parameter changes what the agent learns.

1.7 Chapter Review

Reinforcement learning is the problem of an agent learning, by interaction, what to do in order to maximize a numerical reward signal. It differs from supervised learning (no labeled examples) and unsupervised learning (there is an objective). The defining feature is that the agent's actions affect the data it gets to learn from — and so the agent must trade off *exploiting* apparently-best actions against *exploring* other actions to gather more information.

An RL system is built out of an agent and an environment, plus up to four sub-elements: a *policy* (how to act), a *reward signal* (the immediate goal), a *value function* (the long-run goal), and optionally a *model* of the environment (used for planning). Most modern RL algorithms estimate value functions from experience and use them to improve the policy.

A simple example using tic-tac-toe shows how a value-function method can learn to play a game well without any prior knowledge of strategy, just by repeatedly playing and updating value estimates with a temporal-difference rule. The next chapter takes this idea further with Q-learning on the game of blackjack.

1.8 Exercises

Exercise 1.1. Classify each of the following as best fitting supervised, unsupervised, or reinforcement learning, and briefly justify:

- A spam filter trained on a labeled corpus of spam and non-spam email.
- An autonomous vacuum that learns over time which rooms are dirtiest at which times of day and adjusts its schedule accordingly.

- A program that finds groupings ("communities") in a social network from connection data alone.
- A computer program that learns to play chess by playing many games against itself.

Exercise 1.2. Suppose you are designing a reward signal for an RL agent that drives a delivery robot. Sketch a reward signal that would encourage the robot to deliver packages quickly without colliding with obstacles. Then describe a way that a poorly-chosen reward could cause the robot to "game" the system instead of doing what you intended.

Exercise 1.3. The ε-greedy strategy uses a single parameter ε for the probability of taking a random action. What are the consequences of choosing ε too large? Too small? Suggest a sensible way to schedule ε over the course of training.